

บทที่ 2

ทฤษฎีที่เกี่ยวข้อง

ในการดำเนินโครงการระบบเฝ้าระวังและแจ้งเตือนภัยพิบัติอัจฉริยะ กรณีศึกษา คณะผู้จัดทำได้ค้นคว้าและรวบรวมข้อมูลที่เกี่ยวข้องเพื่อใช้เป็นพื้นฐานในการออกแบบและพัฒนาระบบเฝ้าระวังแจ้งเตือน และบริการผู้ช่วยด้วยเทคโนโลยีเว็บและปัญญาประดิษฐ์สมัยใหม่ โดยทำการศึกษาข้อมูลตามลำดับดังนี้

- 2.1 แนวคิดเกี่ยวกับภัยพิบัติและการจัดการภัยพิบัติ
- 2.2 ระบบเฝ้าระวังภัยพิบัติอัจฉริยะ
- 2.3 เทคโนโลยีที่ใช้ในระบบดี-มายด์
- 2.4 โปรโตคอลที่ใช้ในการสื่อสาร
- 2.5 ภาษาที่ใช้ในการเขียนโปรแกรม
- 2.6 ระบบฐานข้อมูล
- 2.7 ปัญญาประดิษฐ์
- 2.8 ระบบสารสนเทศภูมิศาสตร์
- 2.9 งานวิจัยที่เกี่ยวข้อง

2.1 แนวคิดเกี่ยวกับภัยพิบัติและการจัดการภัยพิบัติ

การบริหารจัดการภัยพิบัติเป็นกระบวนการดำเนินงานที่เป็นวงจรต่อเนื่อง ครอบคลุมตั้งแต่ช่วงก่อนเกิดเหตุ ระหว่างเกิดเหตุและหลังเกิดเหตุ เพื่อลดความสูญเสียต่อชีวิตและทรัพย์สิน โดยทั่วไปสามารถแบ่งวงจรการจัดการภัยพิบัติออกเป็น 4 ระยะสำคัญ ได้แก่ 1) ระยะการเฝ้าระวังและลดผลกระทบ 2) ระยะการเตรียมความพร้อม 3) ระยะการตอบโต้ภาวะฉุกเฉิน และ 4) ระยะการฟื้นฟูเยียวยา ซึ่งการดำเนินงานในแต่ละระยะจำเป็นต้องอาศัยข้อมูลสถานการณ์ที่ถูกต้อง แม่นยำ และเป็นปัจจุบัน รวมถึงต้องมีช่องทางการสื่อสารที่มีประสิทธิภาพเพื่อสนับสนุนการตัดสินใจของผู้เกี่ยวข้อง[3]

สำหรับในประเทศไทย การบริหารจัดการสาธารณภัยดำเนินงานโดยมีหน่วยงานหลัก อาทิ กรมป้องกันและบรรเทาสาธารณภัย (ปภ.) กรมอุตุนิยมวิทยา กรมทรัพยากรน้ำ และหน่วยงานด้านธรณีวิทยา ซึ่งทำหน้าที่ในการเฝ้าระวังติดตามสถานการณ์ วิเคราะห์ข้อมูลเชิงวิชาการ และออกประกาศแจ้งเตือนภัยต่อสาธารณชนผ่านสื่อประชาสัมพันธ์รูปแบบต่าง ๆ เพื่อให้ประชาชนได้รับทราบข้อมูลข่าวสารอย่างทั่วถึง[4]

แนวโน้มสำคัญของการจัดการภัยพิบัติในยุคดิจิทัล คือการประยุกต์ใช้เทคโนโลยีสารสนเทศ และการสื่อสาร และระบบอัจฉริยะเพื่อสนับสนุนระบบเตือนภัยล่วงหน้า เช่น การใช้งานระบบ เซ็นเซอร์ตรวจวัดระยะไกล เครือข่ายอินเทอร์เน็ตของสรรพสิ่ง แพลตฟอร์มการวิเคราะห์ข้อมูลขนาดใหญ่ รวมถึงแอปพลิเคชันบนอุปกรณ์เคลื่อนที่ซึ่งสามารถส่งการแจ้งเตือนได้แบบเวลาจริง[1] ทั้งนี้ ระบบดี-มายด์ได้ถูกออกแบบและพัฒนาขึ้นโดยสอดคล้องกับแนวคิดดังกล่าว ด้วยการรวบรวมข้อมูล ภัยพิบัติจากฐานข้อมูลภายนอก และข้อมูลสภาพแวดล้อมจากอุปกรณ์ไอโอที เพื่อนำมาประมวลผล และแสดงผลผ่านแผนที่ภูมิสารสนเทศ พร้อมระบบแจ้งเตือนอัจฉริยะ ซึ่งช่วยเพิ่มประสิทธิภาพในการ รับรู้สถานการณ์ล่วงหน้า และลดความเสี่ยงจากภัยพิบัติที่อาจเกิดขึ้น

2.2 ระบบเฝ้าระวังภัยพิบัติอัจฉริยะ

องค์ประกอบสำคัญของการจัดการภัยพิบัติคือ ระบบเตือนภัยล่วงหน้า ซึ่งทำหน้าที่ตรวจจับ สัญญาณความผิดปกติ ประเมินความเสี่ยง และส่งสัญญาณแจ้งเตือนไปยังประชาชนและหน่วยงานที่ เกี่ยวข้องผ่านช่องทางการสื่อสารที่หลากหลาย เช่น สัญญาณเสียงเตือนภัย บริการข้อความสั้น การ กระจายเสียงทางวิทยุและโทรทัศน์ รวมถึงแอปพลิเคชันบนสมาร์ตโฟน [7] ในปัจจุบันมีการใช้งาน แอปพลิเคชันเตือนภัยอย่างแพร่หลายทั้งในระดับสากลและในประเทศไทย เช่น แอปพลิเคชัน "THAI DISASTER ALERT" และ "PhonPhai" (พ่นภัย) ซึ่งมีความสามารถในการแจ้งเตือนสถานการณ์พายุ น้ำท่วม แผ่นดินไหว และฝุ่นละอองขนาดเล็ก (PM2.5) โดยอ้างอิงจากพิกัดตำแหน่งของผู้ใช้งาน ช่วย ให้ประชาชนสามารถเตรียมพร้อมรับมือภัยพิบัติในพื้นที่ของตนได้อย่างรวดเร็ว[8]

แนวคิด "ระบบเฝ้าระวังภัยพิบัติอัจฉริยะ" หมายถึงระบบที่ผสมผสานการทำงานของ เซ็นเซอร์อัตโนมัติ การสื่อสารข้อมูลแบบเรียลไทม์ การวิเคราะห์ข้อมูลขั้นสูง และส่วนติดต่อผู้ใช้งานที่ เข้าใจง่าย เพื่อสร้างความตระหนักรู้ต่อสถานการณ์ ผู้วิจัยจึงได้อ้างอิงคุณลักษณะเด่น 2 ประการ ได้แก่ 1) การแสดงผลข้อมูลเชิงพื้นที่บนแผนที่ภูมิสารสนเทศ 2) การแจ้งเตือนภัยแบบเฉพาะเจาะจง พื้นที่และระดับความรุนแรง[9]

จากแนวคิดและเทคโนโลยีดังกล่าว ระบบดี-มายด์ ได้นำมาประยุกต์ใช้ในการออกแบบ สถาปัตยกรรมระบบ เพื่อรวบรวมข้อมูลจากเอพีไอ สาธารณะและอุปกรณ์ไอโอที มาแสดงผลผ่านเว็บ และแอปพลิเคชัน พร้อมฟังก์ชันการแจ้งเตือนภัยพิบัติแก่ผู้ใช้งานในพื้นที่เสี่ยง ซึ่งจะช่วยยกระดับ ประสิทธิภาพการเฝ้าระวังและบริหารจัดการภัยพิบัติในบริบทของประเทศไทยให้ดียิ่งขึ้น

2.3 เทคโนโลยีที่ใช้ในระบบดี-มายด์

ระบบดี-มายด์ ได้รับการออกแบบภายใต้สถาปัตยกรรมที่บูรณาการเทคโนโลยีสำคัญ 4 ส่วน ได้แก่ เทคโนโลยีอินเทอร์เน็ตของสรรพสิ่ง การเชื่อมต่อข้อมูลผ่านส่วนต่อประสานโปรแกรมประยุกต์ ระบบฐานข้อมูลบนคลาวด์ และแอปพลิเคชันบนอุปกรณ์เคลื่อนที่ เพื่อให้ระบบสามารถรองรับการเฝ้าระวังและแจ้งเตือนภัยพิบัติได้ในลักษณะใกล้เคียงเวลาจริง การออกแบบโครงสร้างระบบในลักษณะแยกส่วนช่วยให้กระบวนการพัฒนาและบำรุงรักษามีความยืดหยุ่นสูง สามารถปรับปรุงหรือขยายขีดความสามารถของแต่ละส่วนได้โดยไม่ส่งผลกระทบต่อภาพรวมของระบบ[10]

2.3.1 เทคโนโลยีอินเทอร์เน็ตของสรรพสิ่ง (Internet of Things: IoT)

ในส่วนของอุปกรณ์ตรวจวัดภาคสนาม ระบบดี-มายด์มุ่งเน้นการประยุกต์ใช้อุปกรณ์เซ็นเซอร์และไมโครคอนโทรลเลอร์เพื่อตรวจวัดค่าตัวแปรทางสภาพแวดล้อมที่สำคัญ อาทิ ระดับน้ำ ปริมาณน้ำฝน ปริมาณฝุ่นละออง และการสั่นสะเทือนของพื้นดิน ข้อมูลที่ได้จากการตรวจวัดจะถูกส่งผ่านเครือข่ายสื่อสารไปยังเซิร์ฟเวอร์อย่างต่อเนื่อง การนำข้อมูลจากเซ็นเซอร์ในพื้นที่จริงมาประมวลผลร่วมกับข้อมูลจากหน่วยงานส่วนกลาง ช่วยเติมเต็มช่องว่างของข้อมูลและเพิ่มขีดความสามารถในการตรวจจับความผิดปกติในระดับชุมชน ทำให้การกำหนดเกณฑ์การแจ้งเตือนมีความสอดคล้องกับบริบทของพื้นที่มากยิ่งขึ้น[11] อุปกรณ์ฮาร์ดแวร์ที่ใช้มีดังนี้

2.3.1.1 ราสเบอร์รี่พาย (Raspberry Pi)

Raspberry Pi 4 เป็นไมโครคอมพิวเตอร์บอร์ดเดี่ยวที่มีประสิทธิภาพการประมวลผลสูงขึ้นเมื่อเทียบกับรุ่นก่อนหน้า โดยในงานวิจัยนี้เลือกใช้รุ่น Raspberry Pi 4 Model B ซึ่งใช้หน่วยประมวลผล Broadcom BCM2711 Quad-core Cortex-A72 (ARM v8) 64-bit ความเร็ว 1.5GHz อุปกรณ์รุ่นนี้รองรับการเชื่อมต่อที่ทันสมัย อาทิ พอร์ต USB 3.0 สำหรับการถ่ายโอนข้อมูลความเร็วสูงและการเชื่อมต่อไร้สายผ่าน Dual-band Wi-Fi และ Bluetooth 5.0 ด้วยความสามารถในการประมวลผลที่ใกล้เคียงกับคอมพิวเตอร์ตั้งโต๊ะขนาดเล็กทำให้ Raspberry Pi 4 ทำหน้าที่เป็นหน่วยประมวลผลกลางของสถานีตรวจวัดในระบบดี-มายด์เพื่อทำหน้าที่รวบรวมข้อมูลจากโหนดเซ็นเซอร์และบริหารจัดการการส่งข้อมูลไปยังระบบคลาวด์ได้อย่างมีประสิทธิภาพ[49]

2.3.1.2 อีเอสพี-32 (ESP-32)

อีเอสพี-32 เป็นไมโครคอนโทรลเลอร์ประเภท System on a Chip (SoC) ที่ได้รับความนิยมสูงในงานด้านอินเทอร์เน็ตของสรรพสิ่ง (IoT) เนื่องจากมีโมดูลสื่อสารไร้สายทั้ง Wi-Fi และ Bluetooth ติดตั้งมาในตัวชิป คุณลักษณะเด่นของอีเอสพี-32 คือการใช้หน่วยประมวลผลแบบ Dual-core สถาปัตยกรรม Xtensa® Dual-Core 32-bit LX6 ซึ่งมีความเร็วในการทำงานสูงสุดถึง 240MHz พร้อมทั้งมีโหมดประหยัดพลังงานที่หลากหลาย นอกจากนี้ยังมีพอร์ตเชื่อมต่อ (GPIO) จำนวนมากที่รองรับการใช้งานร่วมกับเซ็นเซอร์หลายประเภทผ่านโปรโตคอลมาตรฐาน เช่น

I2C SPI และ UART จึงเหมาะสำหรับการนำมาใช้เป็นสถานีตรวจวัดย่อยในพื้นที่ต่าง ๆ เพื่อส่งค่าตัวแปรทางสภาพแวดล้อมกลับมายังระบบส่วนกลาง[50]

2.3.1.3 โมดูลไจโรสโคปและเครื่องวัดความเร่ง (GY-521 / MPU-6050)

โมดูล GY-521 เป็นอุปกรณ์ตรวจวัดการเคลื่อนไหวที่มีชิปหลักคือ MPU6050 ซึ่งรวมเอาเซ็นเซอร์วัดความเร่งแบบ 3 แกน และเซ็นเซอร์วัดความเร็วเชิงมุมแบบ 3 แกนไว้ในตัวเดียวกัน รวมเป็น 6 แกน อุปกรณ์นี้ใช้การประมวลผลสัญญาณแบบดิจิทัลภายใน ช่วยลดภาระการคำนวณของไมโครคอนโทรลเลอร์และสื่อสารผ่านโปรโตคอล I2C ในโครงงานนี้ได้นำ GY-521 มาใช้เพื่อตรวจวัดการเอียงตัวของชั้นดินซึ่งเป็นข้อมูลสำคัญในการเฝ้าระวังภัยพิบัติประเภทแผ่นดินไหวหรือดินถล่มเพื่อให้ระบบสามารถแจ้งเตือนสถานการณ์ความผิดปกติได้อย่างแม่นยำ[51]

2.3.1.4 เซ็นเซอร์วัดอุณหภูมิ ความชื้น และความกดอากาศ (BME-280)

BME-280 เป็นโมดูลเซ็นเซอร์ตรวจวัดสภาพแวดล้อมที่สามารถวัดค่าความชื้นสัมพัทธ์ ความกดอากาศและอุณหภูมิได้พร้อมกันในชิปเดียว อุปกรณ์นี้ถูกออกแบบมาเพื่อการใช้งานที่ต้องการการประหยัดพลังงานเป็นหลักโดยรองรับการเชื่อมต่อผ่านโปรโตคอลมาตรฐานทั้ง I2C และ SPI ในระบบตี-มายด์ BME-280 ทำหน้าที่ให้ข้อมูลสภาพอากาศพื้นฐานในพื้นที่ติดตั้งเพื่อนำมาวิเคราะห์ความสัมพันธ์กับสถานการณ์ภัยพิบัติในระดับจุลภาค[52]

2.3.1.5 เซ็นเซอร์วัดระยะทางแบบอัลตราโซนิก (AJ-SR04M)

AJ-SR04M เป็นเซ็นเซอร์อัลตราโซนิกสำหรับวัดระยะทางที่มีความโดดเด่นด้านความทนทาน เนื่องจากตัวหัวตรวจวัดถูกออกแบบมาให้มีคุณสมบัติกันน้ำ หลักการทำงานอาศัยการส่งคลื่นเสียงความถี่สูงและจับเวลาการสะท้อนกลับเพื่อคำนวณระยะห่างของวัตถุโดยมีระยะการทำงานที่มีประสิทธิภาพตั้งแต่ 20 เซนติเมตร ถึง 450 เซนติเมตร เซ็นเซอร์ชนิดนี้มีความเหมาะสมอย่างยิ่งสำหรับการติดตั้งภาคสนามในโครงการตี-มายด์ เพื่อใช้ตรวจวัดระดับน้ำในแหล่งน้ำธรรมชาติหรือพื้นที่เสี่ยงภัยเพื่อเฝ้าระวังสถานการณ์น้ำท่วม[53]

2.3.1.6 เซ็นเซอร์วัดคุณภาพอากาศ (PMS5003)

PMS5003 เป็นเซ็นเซอร์สำหรับตรวจวัดความเข้มข้นของฝุ่นละอองในอากาศโดยใช้หลักการกระเจิงของแสงเลเซอร์ (Laser Scattering)[55] อุปกรณ์นี้สามารถตรวจจับและรายงานค่าความเข้มข้นของอนุภาคที่มีขนาดเล็กได้หลายระดับ ได้แก่ PM1.0 PM2.5 และ PM10 แบบเรียลไทม์ผ่านส่วนต่อประสาน UART ด้วยความแม่นยำในการตรวจวัดอนุภาคที่เล็กถึง 0.3 ไมครอน ทำให้ระบบตี-มายด์สามารถใช้ข้อมูลดังกล่าวในการแจ้งเตือนปัญหามลพิษทางอากาศและความเสี่ยงจากหมอกควันแก่ผู้ใช้งานได้[54]

2.3.2 การเชื่อมต่อเอพีไอข้อมูลภัยพิบัติจากแหล่งข้อมูลแบบเปิด

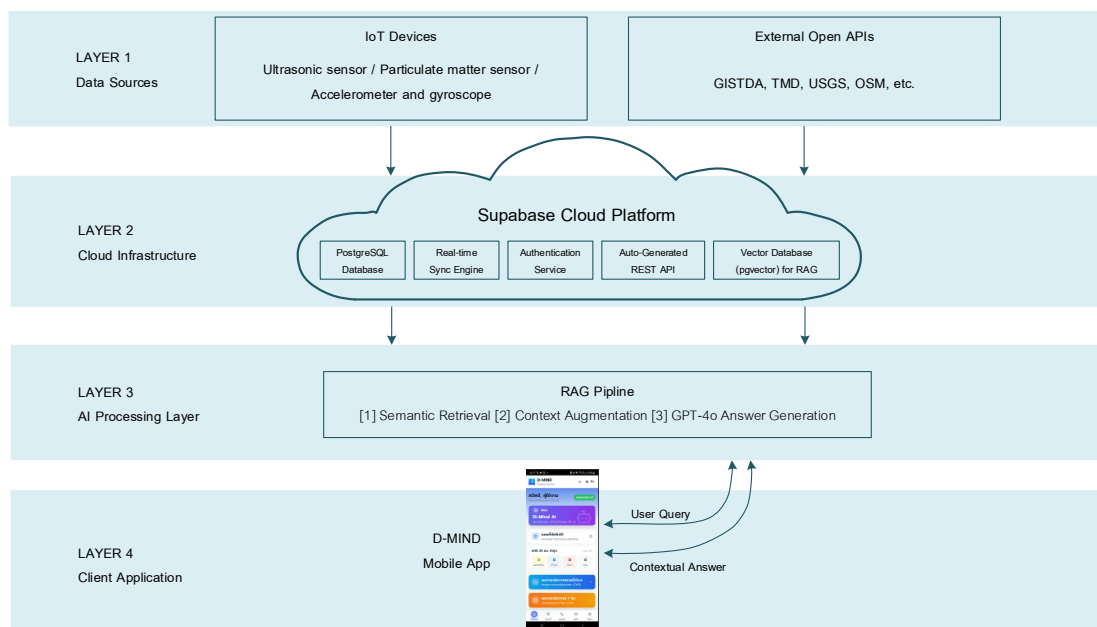
นอกเหนือจากข้อมูลที่ได้จากอุปกรณ์ตรวจวัดภาคสนาม ระบบดี-มายด์ยังถูกออกแบบให้สามารถดึงข้อมูลจากแหล่งข้อมูลภายนอกผ่านเอพีไอ เช่น บริการข้อมูลแผ่นดินไหว ซึ่งให้รายละเอียดเกี่ยวกับเวลาที่เกิดเหตุ พิกัดละติจูด-ลองจิจูด และขนาดความรุนแรง การเชื่อมต่อกับเอพีไอเหล่านี้ช่วยให้ระบบสามารถบูรณาการข้อมูลจากเครือข่ายตรวจวัดระดับประเทศและระดับโลก เข้ากับข้อมูลภายในระบบ ส่งผลให้การวิเคราะห์และแจ้งเตือนภัยมีความครอบคลุมและน่าเชื่อถือมากยิ่งขึ้น[12]

2.3.3 ระบบฐานข้อมูลบนคลาวด์

สำหรับการจัดการข้อมูล ผู้วิจัยเลือกใช้แพลตฟอร์มฐานข้อมูลบนคลาวด์ที่มีชื่อว่า Supabase ซึ่งพัฒนามาบนพื้นฐานของระบบจัดการฐานข้อมูล PostgreSQL โดยมีจุดเด่นด้านฟังก์ชันการยืนยันตัวตน การสร้างเอพีไออัตโนมัติ และความสามารถในการซิงโครไนซ์ข้อมูลแบบเรียลไทม์ โครงสร้างฐานข้อมูลได้รับการออกแบบให้รองรับการจัดเก็บข้อมูลที่มีความซับซ้อน ทั้งข้อมูลเหตุการณ์ภัยพิบัติ ข้อมูลผู้ใช้งาน ประวัติการแจ้งเตือน และข้อมูลอนุกรมเวลาจากอุปกรณ์ไอโอที โดยสามารถขยายตัวระบบในอนาคตได้[13][14]

2.3.4 แอปพลิเคชันมือถือ

ส่วนติดต่อผู้ใช้งานของระบบดี-มายด์ ได้รับการพัฒนาจากเว็บแอปพลิเคชันเดิมมาในรูปแบบแอปพลิเคชันบนมือถือ เพื่ออำนวยความสะดวกให้ผู้ใช้งานสามารถเข้าถึงข้อมูลสถานการณ์ภัยพิบัติบนแผนที่ภูมิสารสนเทศ รับการแจ้งเตือน รายงานเหตุการณ์ฉุกเฉิน และศึกษาคู่มือการปฏิบัติตนเมื่อเกิดภัย[15] การเชื่อมต่อแอปพลิเคชันกับฐานข้อมูลบนคลาวด์และเอพีไอจากแหล่งข้อมูลภายนอกช่วยให้ผู้ใช้งานได้รับข้อมูลที่เป็นปัจจุบันอยู่เสมอ อีกทั้งสถาปัตยกรรมระบบยังสามารถรองรับปริมาณผู้ใช้งานจำนวนมากได้โดยไม่กระทบต่อประสิทธิภาพการทำงาน[16]



ภาพที่ 2.1 ไดอะแกรมภาพรวมเทคโนโลยีที่ใช้ในระบบ

2.3.5 การพัฒนาแอปพลิเคชันแอนดรอยด์แบบเนทีฟ

การพัฒนาแอปพลิเคชันแอนดรอยด์แบบเนทีฟ เป็นแนวทางการพัฒนาแอปพลิเคชันที่ใช้เครื่องมือ ภาษาโปรแกรม และชุดคำสั่งที่ออกแบบมาสำหรับระบบปฏิบัติการแอนดรอยด์โดยตรง เช่น ภาษาโคทลิน ภาษาจาวา และ Android SDK ส่งผลให้แอปพลิเคชันสามารถทำงานร่วมกับความสามารถของอุปกรณ์เคลื่อนที่ได้อย่างเต็มประสิทธิภาพ ไม่ว่าจะเป็นการเข้าถึงตำแหน่งของผู้ใช้ การทำงานเบื้องหลัง การแจ้งเตือนแบบเนทีฟ การจัดการสิทธิ์การใช้งานของระบบ และการเชื่อมต่อกับบริการของระบบปฏิบัติการ

ในงานวิจัยนี้ ระบบดี-มายด์ได้ต่อยอดจากเว็บแอปพลิเคชันเดิมมาสู่แอปพลิเคชันบนอุปกรณ์เคลื่อนที่ โดยปรับแนวทางการพัฒนาจากการใช้เว็บเทคโนโลยีร่วมกับตัวกลางสำหรับเชื่อมต่อแพลตฟอร์มมือถือไปเป็นการพัฒนาแบบเนทีฟบนระบบปฏิบัติการแอนดรอยด์เพื่อให้สามารถรองรับฟังก์ชันที่เกี่ยวข้องกับการแจ้งเตือนภัยพิบัติและการทำงานภาวะฉุกเฉินได้เหมาะสมมากยิ่งขึ้น แอปพลิเคชันจึงถูกพัฒนาด้วยภาษาโคทลินร่วมกับ Jetpack Compose สำหรับสร้างส่วนติดต่อผู้ใช้ และยังคงใช้โค้ดภาษาจาวาบางส่วนในบริการที่เกี่ยวข้องกับระบบปฏิบัติการ เช่น การติดตามตำแหน่งเบื้องหลัง การตรวจจับพื้นที่เสี่ยง การรับข้อความแจ้งเตือนจาก Firebase Cloud Messaging และการทำงานของ WorkManager[5]



ภาพที่ 2.2 แนวทางการพัฒนาแอปพลิเคชันแอนดรอยด์แบบเนทีฟ[5]

2.3.6 เจ็ตแพ็กคอมโพส

เจ็ตแพ็กคอมโพส (Jetpack Compose) เป็นเครื่องมือสำหรับพัฒนาส่วนติดต่อผู้ใช้ของแอปพลิเคชันแอนดรอยด์สมัยใหม่ โดยใช้แนวคิดการสร้างหน้าจอแบบประกาศลักษณะของหน้าจอที่ต้องการแสดงผล หรือ Declarative UI แทนการออกแบบหน้าจอด้วยไฟล์ XML แบบเดิม นักพัฒนาสามารถเขียนส่วนติดต่อผู้ใช้ด้วยภาษาโคทลินได้โดยตรง ทำให้โครงสร้างของหน้าจอมีความกระชับ อ่านเข้าใจง่าย และสามารถเชื่อมโยงกับสถานะของข้อมูลภายในแอปพลิเคชันได้อย่างเป็นระบบ แนวคิดสำคัญของเจ็ตแพ็กคอมโพสคือการแบ่งส่วนติดต่อผู้ใช้ออกเป็นคอมโพเนนต์ย่อยที่เรียกว่า Composable Function ซึ่งแต่ละส่วนทำหน้าที่แสดงผลองค์ประกอบของหน้าจอ เช่น ปุ่ม ข้อความ การ์ด แถบนำทาง รายการข้อมูล หรือกล่องแสดงสถานะ เมื่อข้อมูลหรือสถานะของระบบเปลี่ยนแปลง คอมโพสจะปรับปรุงหน้าจอเฉพาะส่วนที่เกี่ยวข้องโดยอัตโนมัติ ช่วยลดความซับซ้อนในการจัดการหน้าจอและลดโอกาสเกิดข้อผิดพลาดจากการควบคุมส่วนติดต่อผู้ใช้ด้วยตนเอง[17]

ในระบบดี-มายด์ เจ็ตแพ็กคอมโพสถูกนำมาใช้เป็นเครื่องมือหลักในการพัฒนาส่วนติดต่อผู้ใช้ของแอปพลิเคชันแอนดรอยด์แบบเนทีฟ โดยใช้สำหรับสร้างโครงสร้างหน้าจอหลักของแอปพลิเคชัน เช่น หน้าหลักสำหรับแสดงความพร้อมของระบบ หน้าจอแผนที่ภัยพิบัติ หน้าจอระบบแจ้งเตือน หน้าจอขอความช่วยเหลือฉุกเฉิน และหน้าจอฟังก์ชันเพิ่มเติม การออกแบบดังกล่าวช่วยให้แอปพลิเคชันสามารถจัดวางองค์ประกอบบนหน้าจอให้เหมาะสมกับการใช้งานบนอุปกรณ์เคลื่อนที่ และรองรับการขยายฟีเจอร์ในอนาคตได้สะดวก นอกจากนี้ ระบบดี-มายด์ยังใช้เจ็ตแพ็กคอมโพสร่วมกับ Material 3 เพื่อกำหนดรูปแบบของหน้าจอ สี ธีม และองค์ประกอบส่วน

ติดต่อผู้ใช้ให้มีความสอดคล้องกันทั้งระบบ เช่น การออกแบบแถบนำทางด้านล่าง การแสดงการ์ดสถานะ การใช้ปุ่มปฏิบัติการหลัก และการกำหนดโทนสีของแอปพลิเคชันให้เหมาะสมกับระบบแจ้งเตือนภัยพิบัติที่ต้องการความชัดเจนในการสื่อสารกับผู้ใช้งาน

2.3.7 แมปลิเบรและโอเพนสตรีตแมปสำหรับแผนที่ภัยพิบัติ

แมปลิเบร (MapLibre) เป็นไลบรารีสำหรับแสดงผลแผนที่เชิงโต้ตอบที่สามารถนำมาใช้งานร่วมกับข้อมูลเชิงพื้นที่ในรูปแบบต่าง ๆ เช่น จุดพิกัด เส้นทาง พื้นที่เสี่ยง และชั้นข้อมูลซ้อนทับบนแผนที่ ในระบบดี-มายด์ได้นำแมปลิเบรมาใช้เป็นองค์ประกอบหลักของหน้าจอแผนที่ภัยพิบัติบนแอปพลิเคชันแอนดรอยด์แบบเนทีฟเพื่อแสดงตำแหน่งเหตุการณ์หรือข้อมูลภัยพิบัติในลักษณะมาร์คเกอร์และโอเวอร์เลย์บนแผนที่โดยใช้โอเพนสตรีตแมปเป็นแผนที่ฐานในรูปแบบ Raster Tile ทำให้ผู้ใช้สามารถมองเห็นตำแหน่งของภัยพิบัติและพื้นที่เสี่ยงในบริบทของภูมิประเทศจริงได้อย่างชัดเจน

การใช้แมปลิเบรร่วมกับโอเพนสตรีตแมปมีความเหมาะสมกับระบบเฝ้าระวังภัยพิบัติเนื่องจากสามารถรองรับการแสดงผลแผนที่แบบเต็มหน้าจอ การควบคุมการซูม การระบุตำแหน่งผู้ใช้ การแสดงเหตุผลเหตุการณ์ และการซ้อนทับข้อมูลจากหลายแหล่งได้ ในระบบดี-มายด์ หน้าจอแผนที่ถูกออกแบบให้มีส่วนประกอบสำคัญ เช่น แถบค้นหา ปุ่มควบคุมแผนที่ ปุ่มกรองข้อมูล Bottom Sheet สำหรับสรุปสถานการณ์ และการแสดงสถิติภัยพิบัติ เพื่อให้ผู้ใช้สามารถติดตามสถานการณ์ได้สะดวกและเข้าใจง่ายมากขึ้น[24]

2.3.8 ระบบแจ้งเตือนบนแอนดรอยด์และไฟร์เบสคลาวด์เมสเซจจิง

ระบบแจ้งเตือนบนแอนดรอยด์เป็นองค์ประกอบสำคัญของแอปพลิเคชันเฝ้าระวังภัยพิบัติ เนื่องจากต้องสามารถแจ้งเตือนผู้ใช้เมื่อเกิดเหตุการณ์สำคัญได้อย่างรวดเร็วและน่าเชื่อถือ ระบบดี-มายด์ใช้ไฟร์เบสคลาวด์เมสเซจจิงเป็นช่องทางในการส่งข้อความแจ้งเตือนจากฝั่งเซิร์ฟเวอร์ไปยังอุปกรณ์แอนดรอยด์ของผู้ใช้ โดยแอปพลิเคชันจะลงทะเบียน FCM token และส่งไปยังแบคเอนด์ผ่านเอนพอยต์ /fcm/register จากนั้นแบคเอนด์สามารถส่งข้อความแจ้งเตือนกลับมายังอุปกรณ์ผ่าน FCM HTTP v1 ได้

เมื่ออุปกรณ์ได้รับข้อความแจ้งเตือน แอปพลิเคชันจะประมวลผลข้อมูลที่อยู่ในเพย์โหลดเช่น alert_type, alert_title และ alert_message แล้วนำไปแสดงผลผ่านระบบแจ้งเตือนแบบเนทีฟของแอนดรอยด์โดยใช้ Notification channel Vibration และ Full-screen intent สำหรับกรณีฉุกเฉิน ระบบนี้ช่วยให้แอปพลิเคชันสามารถแจ้งเตือนภัยพิบัติได้แม้ผู้ใช้ไม่ได้เปิดแอปอยู่ และสามารถแสดงการแจ้งเตือนแบบเต็มหน้าจอในสถานการณ์ที่มีความสำคัญสูง เช่น ภัยพิบัติรุนแรงหรือเหตุฉุกเฉินที่ต้องการให้ผู้ใช้รับทราบทันที

2.3.9 การจัดเก็บข้อมูลภายในอุปกรณ์ด้วย SQLite และ Room Database

การจัดเก็บข้อมูลภายในอุปกรณ์เป็นส่วนสำคัญที่ช่วยให้แอปพลิเคชันสามารถทำงานได้ต่อเนื่องแม้ในสภาพแวดล้อมที่เครือข่ายไม่เสถียร ระบบดี-มายด์มีการใช้ฐานข้อมูลภายในอุปกรณ์เพื่อจัดเก็บข้อมูลที่เกี่ยวข้องกับการแจ้งเตือน พื้นที่เสี่ยง ตำแหน่งของผู้ใช้ และขอความช่วยเหลือฉุกเฉิน โดยในโครงสร้างเดิมมีการใช้ SQLite สำหรับ Local cache และ Queue DAO ส่วนโครงสร้างใหม่ได้เตรียม Room Database เป็นแนวทางสำหรับการจัดการข้อมูลภายในแอปอย่างเป็นระบบมากขึ้น

Room Database เป็นไลบรารีของแอนดรอยด์ที่ช่วยให้การจัดการฐานข้อมูล SQLite มีความปลอดภัยและเป็นระเบียบมากขึ้น โดยสามารถกำหนด Schema Entity และ Data access object ได้อย่างชัดเจน ในระบบดี-มายด์ Room ถูกออกแบบให้รองรับข้อมูลสำคัญ เช่น Alerts SOS queue Danger zones และ Location history เพื่อให้ระบบสามารถบันทึกสถานะที่จำเป็นไว้ในเครื่องและนำกลับมาใช้งานได้เมื่อผู้ใช้กลับมาเชื่อมต่อเครือข่ายหรือเมื่อระบบต้องตรวจสอบพื้นที่เสี่ยงซ้ำในภายหลัง

2.3.10 โมบายแบ็กเอนด์ด้วย Ktor

เคเทอร์ (Ktor) เป็นเฟรมเวิร์กสำหรับพัฒนาเว็บเซิร์ฟเวอร์และเอพีไอด้วยภาษาโคทลินโดยมีจุดเด่นด้านความยืดหยุ่น น้ำหนักเบา และเหมาะกับการพัฒนาแบคเอนด์ที่ต้องให้บริการกับแอปพลิเคชันมือถือ ในระบบดี-มายด์มีการแยกแบคเอนด์ออกมาเป็นโมดูล android/backend เพื่อทำหน้าที่เป็นโมบายด์แบคเอนด์สำหรับให้บริการเอพีไอที่เกี่ยวข้องกับแอปพลิเคชัน เช่น /health /alerts /sos /reports /weather /damage-assessment /chat และ /fcm/register

การใช้ Ktor เป็นแบคเอนด์ช่วยแยกงานที่เกี่ยวข้องกับการเชื่อมต่อนอก และการจัดการข้อมูลที่มีความอ่อนไหวออกจากตัวแอปพลิเคชันมือถือ เช่น การเรียกใช้บริการเอไอ การประเมินความเสียหาย การส่งข้อมูล SOS การจัดการ FCM token และการเชื่อมต่อเอพีไอภายนอก โดยค่าความลับหรือ API key สำหรับโปรดักชันควรถูกจัดเก็บไว้ใน Environment variables หรือ Secret manager ฝั่งเซิร์ฟเวอร์ ไม่ควรฝังไว้ในตัวแอปพลิเคชันโดยตรง แนวทางนี้ช่วยเพิ่มความปลอดภัยและทำให้ระบบสามารถขยายบริการในอนาคตได้สะดวกมากขึ้น

2.3.11 เทคโนโลยีเว็บแอปพลิเคชันเดิม: วิท รีแอคไทป์สคริปต์ และ Leaflet.js

ก่อนการปรับสถาปัตยกรรมสู่แอปพลิเคชันแอนดรอยด์แบบเนทีฟ ระบบดี-มายด์มีพื้นฐานจากเว็บแอปพลิเคชันที่พัฒนาด้วยเทคโนโลยีส่วนหน้า ได้แก่ วิท (Vite) รีแอคไทป์สคริปต์ (TypeScript) และ Leaflet.js โดยวิททำหน้าที่เป็นเครื่องมือสำหรับสร้างและพัฒนาเว็บแอปพลิเคชันให้มีความรวดเร็วในการเริ่มต้นโปรเจกต์และการทดสอบระหว่างพัฒนา ส่วนรีแอคไทป์

สำหรับสร้างส่วนติดต่อผู้ใช้ในรูปแบบคอมโพเนนต์ และไทป์สคริปต์ช่วยเพิ่มความชัดเจนของชนิดข้อมูล ลดข้อผิดพลาดระหว่างการพัฒนา และทำให้โครงสร้างของซอร์สโค้ดมีความเหมาะสมกับระบบที่มีหลายพีเจียร์ เช่น แผนที่ภัยพิบัติ ระบบแจ้งเตือน รายงานเหตุการณ์ และผู้ช่วยอัจฉริยะ[31]

Leaflet.js ถูกใช้ในบริบทของเว็บแอปพลิเคชันเดิมสำหรับแสดงผลแผนที่เชิงโต้ตอบบนเว็บเบราว์เซอร์ เช่น การแสดงตำแหน่งเหตุการณ์ภัยพิบัติ จุดเสี่ยง และข้อมูลเชิงพื้นที่ต่าง ๆ อย่างไรก็ตาม ในการพัฒนาเวอร์ชันแอนดรอยด์ล่าสุด ระบบได้ปรับจากแนวทางเว็บแอปเดิมไปเป็นเนทีฟแอนดรอยด์แอปโดยใช้โคทลิน เจ็ตแพ็กคอมโพส เมเทอเรียล 3 และ MapLibre เป็นแกนหลักของแอปมือถือ ทำให้ Android build ปัจจุบันไม่พึ่งพา React/Vite runtime แต่ยังคงออกแบบเส้นทางการใช้งานและประสบการณ์ผู้ใช้ให้ใกล้เคียงกับเว็บแอปพลิเคชันเดิมเพื่อให้ผู้ใช้สามารถใช้งานระบบต่อเนื่องได้อย่างคุ้นเคย[33]

2.4 โพรโทคอลที่ใช้ในการสื่อสาร

ในการพัฒนาระบบเฝ้าระวังและแจ้งเตือนภัยพิบัติอัจฉริยะ การเลือกใช้โปรโตคอลการสื่อสารที่มีประสิทธิภาพเป็นปัจจัยสำคัญเพื่อให้มั่นใจว่าข้อมูลจากเซ็นเซอร์จะถูกส่งไปยังฐานข้อมูลและแอปพลิเคชันได้อย่างรวดเร็วและแม่นยำ โดยระบบนี้ได้บูรณาการโปรโตคอลหลัก 2 รูปแบบ คือ MQTT และ HTTP/HTTPS เพื่อทำหน้าที่ในส่วนที่แตกต่างกัน ดังนี้

2.4.1 โพรโทคอล MQTT (Message Queuing Telemetry Transport)

MQTT เป็นโปรโตคอลการสื่อสารที่ออกแบบมาสำหรับการรับส่งข้อมูลในลักษณะอินเทอร์เน็ทของสรรพสิ่งโดยเฉพาะ มีคุณลักษณะเด่นคือเป็นโปรโตคอลที่มีขนาดเล็ก กินทรัพยากรต่ำและมีความรวดเร็วในการส่งข้อมูล

2.4.1.1 รูปแบบการทำงาน MQTT ทำงานบนสถาปัตยกรรมแบบ Publish/Subscribe โดยมีตัวกลางที่เรียกว่าโบรคเกอร์ทำหน้าที่จัดการคิวข้อมูล[56]

2.4.1.2 การประยุกต์ใช้ในระบบดี-มายด์ อีเอสพี-32 ทำหน้าที่เป็น Publisher ส่งข้อมูลจากเซ็นเซอร์ในรูปแบบ JSON Format ไปยังโบรคเกอร์

2.4.2 โพรโทคอล HTTP/HTTPS

HTTP/HTTPS เป็นโปรโตคอลมาตรฐานที่ใช้สำหรับการสื่อสารบนเว็บแอปพลิเคชัน ในงานวิจัยนี้เลือกใช้ HTTPS (Port 443) เป็นหลักเพื่อความปลอดภัยในการรับส่งข้อมูลผ่านเครือข่ายสาธารณะ

2.4.2.1 รูปแบบการทำงาน HTTP/HTTPS ทำงานในลักษณะ Request/Response โดยเครื่องลูกข่ายจะส่งคำร้องขอไปยังเครื่องแม่ข่ายและรอการตอบกลับ[57]

2.4.2.2 การประยุกต์ใช้ในระบบดี-มายด์

1) Application Programming Interface (API) ระบบใช้โปรโตคอลนี้ในการเชื่อมต่อกับแหล่งข้อมูลภายนอกทั้ง 4 แหล่ง ได้แก่ กรมอุตุนิยมวิทยา GISTDA USGS และ OpenStreetMap

2) Database Interaction แอปพลิเคชันบนแอนดรอยด์ใช้ HTTPS ในการดึงข้อมูลเซ็นเซอร์และประวัติเหตุการณ์จากฐานข้อมูลซูบาเบสผ่าน RESTful API

3) Security การใช้ HTTPS ช่วยปกป้องข้อมูลส่วนบุคคลของผู้ใช้งานและประวัติการแจ้งเตือนจากการดักจับข้อมูล

2.5 ภาษาที่ใช้ในการเขียนโปรแกรม

2.5.1 ภาษาไทป์สคริปต์ (TypeScript)

ภาษาไทป์สคริปต์ (TypeScript) เป็นภาษาโปรแกรมแบบโอเพนซอร์สที่พัฒนาโดยบริษัท Microsoft โดยถูกออกแบบให้เป็นส่วนขยายของภาษาจาวาสคริปต์ ซึ่งเพิ่มความสามารถด้านการกำหนดชนิดข้อมูลแบบสแตติก ช่วยให้ผู้พัฒนาสามารถตรวจสอบข้อผิดพลาดของโค้ดได้ตั้งแต่ขั้นตอนการพัฒนา ลดปัญหาที่อาจเกิดขึ้นขณะรันโปรแกรม และทำให้โครงสร้างของซอฟต์แวร์มีความชัดเจนมากขึ้น[20]

ในระบบดี-มายด์ ภาษาไทป์สคริปต์ถูกนำมาใช้ในการพัฒนาเว็บแอปพลิเคชันเดิมร่วมกับเฟรมเวิร์ก React เพื่อสร้างส่วนติดต่อผู้ใช้งาน เช่น หน้าจอสรุปสถานการณ์ ระบบแสดงผลแผนที่ภัยพิบัติ และหน้าจอรายงานเหตุการณ์ การใช้ไทป์สคริปต์ช่วยให้การพัฒนาเว็บแอปพลิเคชันมีความเป็นระเบียบ ลดข้อผิดพลาดจากการจัดการข้อมูลหลายประเภท และเอื้อต่อการบำรุงรักษาระบบในระยะยาว อย่างไรก็ตาม ในการพัฒนาเวอร์ชันแอปพลิเคชันแอนดรอยด์แบบเนทีฟ ระบบได้ปรับมาใช้ภาษาโคทลินเป็นภาษาหลักในการพัฒนาแอปพลิเคชันบนอุปกรณ์เคลื่อนที่

2.5.2 ภาษาโคทลิน (Kotlin)

ภาษาโคทลิน (Kotlin) เป็นภาษาโปรแกรมสมัยใหม่ที่พัฒนาโดย JetBrains และได้รับการสนับสนุนอย่างเป็นทางการสำหรับการพัฒนาแอปพลิเคชันบนระบบปฏิบัติการแอนดรอยด์ ภาษาโคทลินถูกออกแบบให้มีรูปแบบการเขียนที่กระชับ อ่านเข้าใจง่าย และสามารถทำงานร่วมกับภาษาจาวาได้อย่างสมบูรณ์ ทำให้นักพัฒนาสามารถนำไลบรารีหรือโค้ดเดิมที่เขียนด้วยภาษาจาวามาใช้งานร่วมกับโค้ดภาษาโคทลินได้อย่างมีประสิทธิภาพ

คุณลักษณะสำคัญของภาษาโคทลิน คือการรองรับระบบชนิดข้อมูลที่ปลอดภัย โดยเฉพาะการจัดการค่าที่อาจเป็นค่าว่าง หรือ Null Safety ซึ่งช่วยลดข้อผิดพลาดประเภท

Null Pointer Exception ที่มักพบในการพัฒนาแอปพลิเคชัน นอกจากนี้ ภาษาโคทลินยังรองรับการเขียนโปรแกรมเชิงวัตถุและเชิงฟังก์ชัน มีโครงสร้างภาษาที่กระชับ และเหมาะสำหรับการพัฒนาแอปพลิเคชันที่ต้องการความเสถียรและสามารถขยายระบบได้ในอนาคต

ในระบบดี-มายด์ ภาษาโคทลินถูกนำมาใช้เป็นภาษาหลักในการพัฒนาแอปพลิเคชันแอนดรอยด์แบบเนทีฟ โดยทำงานร่วมกับ Jetpack Compose สำหรับสร้างส่วนติดต่อผู้ใช้งาน และใช้ในการพัฒนาโครงสร้างหลักของแอปพลิเคชัน เช่น หน้าหลัก หน้าจอแผนที่ภัยพิบัติ ระบบแจ้งเตือน หน้าขอความช่วยเหลือฉุกเฉิน และส่วนเชื่อมต่อข้อมูลกับแบ็กเอนด์ การเลือกใช้ภาษาโคทลินจึงช่วยให้ระบบดี-มายด์สามารถเข้าถึงความสามารถของระบบปฏิบัติการแอนดรอยด์ได้อย่างเหมาะสม เช่น การจัดการสิทธิ์การใช้งาน การติดตามตำแหน่ง การทำงานเบื้องหลัง และการแจ้งเตือนบนอุปกรณ์เคลื่อนที่

2.6 ระบบฐานข้อมูล

2.6.1 ซูปาเบส (Supabase)

ซูปาเบส เป็นแพลตฟอร์มสำหรับการพัฒนาส่วนหลังของระบบแบบสำเร็จรูปในรูปแบบโอเพนซอร์ส ซึ่งได้รับการออกแบบมาเพื่อเป็นทางเลือกทดแทน Firebase โดยให้บริการระบบจัดการฐานข้อมูล PostgreSQL แบบที่มีการบริหารจัดการให้ พร้อมด้วยชุดเครื่องมือเสริมสำหรับการพัฒนาเว็บและแอปพลิเคชันสมัยใหม่ ประกอบด้วย ระบบยืนยันตัวตน ระบบฐานข้อมูลแบบเรียลไทม์ ระบบจัดเก็บไฟล์ และฟังก์ชันการทำงานฝั่งเซิร์ฟเวอร์[14]

จุดเด่นสำคัญของซูปาเบส คือการอนุญาตให้นักพัฒนาสามารถเข้าถึงขีดความสามารถของฐานข้อมูล PostgreSQL ได้อย่างเต็มรูปแบบ ทั้งการใช้งานคำสั่ง SQL มาตรฐาน การสร้างตาราง ความสัมพันธ์ ดัชนี และการติดตั้งส่วนขยายเพิ่มเติม อาทิ pgvector สำหรับงานด้านปัญญาประดิษฐ์ ในขณะเดียวกัน แพลตฟอร์มยังช่วยลดภาระงานด้านการดูแลรักษาโครงสร้างพื้นฐานด้วยระบบสำรองข้อมูลอัตโนมัติและส่วนติดต่อผู้ใช้งานผ่านเว็บที่ใช้งานง่าย[27]

สำหรับโครงการดี-มายด์ ผู้วิจัยได้เลือกใช้ซูปาเบสเป็นศูนย์กลางในการจัดเก็บและบริหารจัดการข้อมูล โดยพิจารณาจากคุณลักษณะที่สอดคล้องกับความต้องการของระบบ ดังนี้

2.6.1.1 ความสามารถในการอัปเดตข้อมูลแบบเรียลไทม์

ซูปาเบสใช้กลไกการดักจับการเปลี่ยนแปลงข้อมูลจาก PostgreSQL และส่งข้อมูลเหตุการณ์ผ่านโปรโตคอล WebSocket ไปยังเครื่องลูกข่าย คุณสมบัตินี้ช่วยให้หน้าเว็บและแอปพลิเคชันของดี-มายด์สามารถแสดงผลข้อมูลภัยพิบัติที่มีการเปลี่ยนแปลงได้ทันทีโดยไม่ต้องทำการรีเฟรชหน้าจอ[28][29]

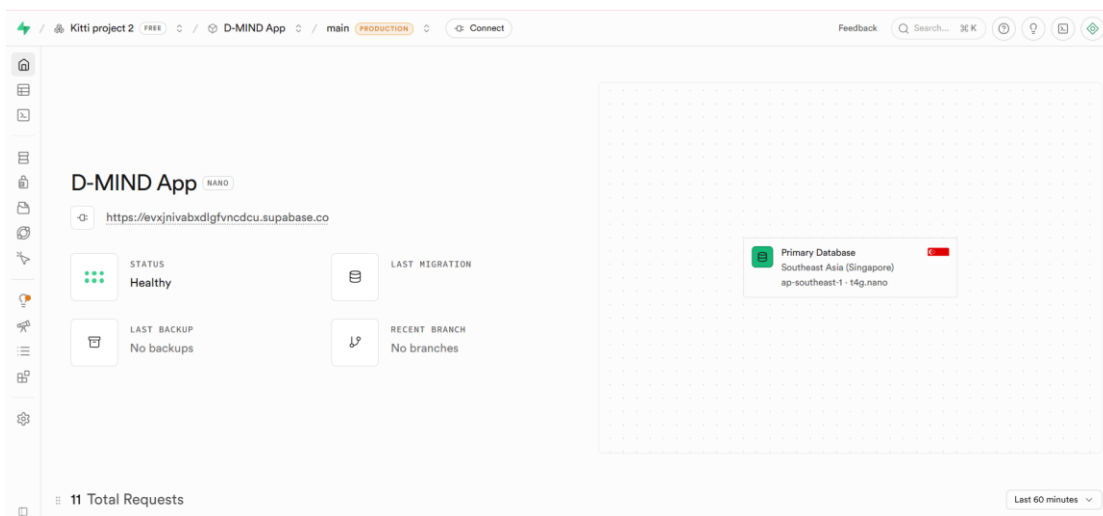
2.6.1.2 ความปลอดภัยระดับแถวข้อมูล

ระบบรองรับการกำหนดนโยบายความปลอดภัยที่ละเอียดถึงระดับแถวข้อมูล ทำให้สามารถควบคุมสิทธิ์การเข้าถึงข้อมูลของผู้ใช้งานแต่ละรายได้อย่างรัดกุม ซึ่งเป็นสิ่งสำคัญสำหรับการคุ้มครองข้อมูลส่วนบุคคลและประวัติการแจ้งเตือน

2.6.1.3 การสร้างเอพีไออัตโนมัติ

แพลตฟอร์มสามารถสร้าง RESTful API จากตารางในฐานข้อมูลได้โดยอัตโนมัติ ช่วยลดระยะเวลาในการพัฒนาส่วนเชื่อมต่อระหว่างฐานข้อมูลและส่วนหน้าของแอปพลิเคชัน[30]

นอกจากนี้ซูพาเบส ยังมีบทบาทสำคัญในการสนับสนุนการทำงานของสถาปัตยกรรม RAG ในระบบผู้ช่วยอัจฉริยะ ผ่านการทำหน้าที่เป็นฐานข้อมูลเวกเตอร์สำหรับจัดเก็บและสืบค้นข้อมูลเชิงความหมาย ซึ่งครอบคลุมทั้งข้อมูลเหตุการณ์ภัยพิบัติ ข้อมูลผู้ใช้งาน และคลังความรู้สำหรับการตอบคำถาม



ภาพที่ 2.3 หน้าต่างระบบฐานข้อมูล Supabase[14]

2.6.2 ฟาสเอพีไอ (fastAPI)

ฟาสเอพีไอ คือเฟรมเวิร์กสมัยใหม่ (Modern Framework) สำหรับการพัฒนาเว็บแอปพลิเคชันในส่วนติดต่อประสานโปรแกรมประยุกต์หรือเอพีไอด้วยภาษาไพทอนที่เน้นประสิทธิภาพและความเร็วในการทำงานสูง โดยในโครงงานนี้ฟาสเอพีไอถูกนำมาประยุกต์ใช้เป็นส่วนประกอบหลักของชั้นตรรกะทางธุรกิจเพื่อทำหน้าที่เป็นตัวกลางในการเชื่อมโยงข้อมูลระหว่าง

อุปกรณ์ไอโอทีและระบบฐานข้อมูล โดยมีรายละเอียดคุณลักษณะที่สำคัญและการประยุกต์ใช้งาน ดังนี้

2.6.2.1 บทบาทหน้าที่ในฐานะตัวกลางเชื่อมโยงข้อมูล (IoT Middleware)

ฟาสเอพีไอทำหน้าที่เป็นส่วนประมวลผลหลักที่ติดตั้งอยู่บนรাসเบอร์รี่พาย เพื่อรับข้อมูลจากชุดเซ็นเซอร์ตรวจวัดภาคสนามที่ส่งผ่านโปรโตคอล MQTT ในรูปแบบเจสันฟอร์มเมต โดยระบบจะทำการแปลงข้อมูลดังกล่าวให้พร้อมสำหรับการบันทึกลงในฐานข้อมูลซูบาเบส ในตาราง `sensor_logs` พร้อมทั้งระบุพิกัดและเวลาที่รับข้อมูลอย่างแม่นยำ[58]

2.6.2.2 การตรวจวัดความผิดปกติและระบบแจ้งเตือนอัตโนมัติ

นอกจากการส่งผ่านข้อมูลแล้วฟาสเอพีไอยังถูกกำหนดให้มีหน้าที่ในการวิเคราะห์ความผิดปกติของข้อมูลในเบื้องต้น ตามเกณฑ์ที่กำหนดในงานวิจัย เช่น การตรวจสอบค่าอุณหภูมิที่สูงเกิน 35 องศาเซลเซียส หรือระดับความชื้นในอากาศที่ผิดปกติ หากระบบตรวจพบเหตุการณ์ที่ไม่เป็นไปตามมาตรฐาน จะดำเนินการส่งคำสั่งแจ้งเตือนไปยังผู้ช่วยอัจฉริยะและระบบแจ้งเตือนในแอปพลิเคชันทันที

2.6.2.3 โครงสร้างจุดเชื่อมต่อบริการ (API Endpoints)

เพื่อให้แอปพลิเคชันดี-มายด์สามารถดึงข้อมูลมาแสดงผลบนแดชบอร์ดและแผนที่ได้อย่างมีประสิทธิภาพฟาสเอพีไอได้ถูกออกแบบให้มีจุดเชื่อมต่อบริการ (Endpoints) ที่สำคัญ ดังนี้:

- 1) GET `/api/sensor-logs` สำหรับการเรียกดูประวัติข้อมูลเซ็นเซอร์ย้อนหลังในรูปแบบการแบ่งหน้า (Pagination)
- 2) GET `/api/sensor-logs/latest` สำหรับดึงข้อมูลล่าสุดจากเซ็นเซอร์เพื่อแสดงผลสถานการณ์แบบเวลาจริง (Real-time Display)
- 3) GET `/api/health` สำหรับตรวจสอบสถานะความพร้อมในการทำงานของระบบ (System Health Check)
- 4) GET `/api/sensors/{sensor_type}` สำหรับกรองข้อมูลเฉพาะประเภทภัยพิบัติ เช่น ข้อมูลฝุ่นละออง PM2.5 หรือระดับน้ำ

2.6.2.4 ประสิทธิภาพและความเสถียรของระบบ

การเลือกใช้ฟาสเอพีไอช่วยให้ระบบดี-มายด์สามารถทำงานแบบไม่พร้อมกัน (Asynchronous Programming) ผ่าน ASGI Server (Uvicorn) ซึ่งส่งผลให้การรับข้อมูลจากเซ็นเซอร์จำนวนมากและการให้บริการแอปพลิเคชันมีข้อดีในเวลาเดียวกันสามารถทำได้โดยไม่มี ความหน่วง นอกจากนี้ยังรองรับการจัดการตัวแปรสภาพแวดล้อมเพื่อเก็บรักษาค่ารหัสผ่านและคีย์สำคัญของซูบาเบสให้มีความปลอดภัยตามมาตรฐานสากล[58]

2.6.2.5 การเชื่อมต่อกับระบบอื่น ๆ (System Integration)

ฟาสเอพีไอทำหน้าที่เป็นศูนย์กลางที่บูรณาการเทคโนโลยีทั้งหมดเข้าด้วยกันตั้งแต่การรับข้อมูลจาก MQTT Broker การจัดเก็บข้อมูลลงใน PostgreSQL ของซูปาเบส ทำให้สถาปัตยกรรมของดี-มายด์มีความยืดหยุ่นและรองรับการขยายขีดความสามารถในอนาคต

2.7 ปัญญาประดิษฐ์

ปัญญาประดิษฐ์ได้เข้ามามีบทบาทสำคัญในการพัฒนาระบบจัดการข้อมูลและสนับสนุนการตัดสินใจในหลากหลายโครงการโดยเฉพาะด้านการเฝ้าระวังและแจ้งเตือนภัยพิบัติ ซึ่งต้องการการประมวลผลข้อมูลแบบรวดเร็วควบคู่ไปกับการให้คำแนะนำที่ถูกต้องและทันท่วงที ในปัจจุบันเทคโนโลยีปัญญาประดิษฐ์ โดยเฉพาะอย่างยิ่งโมเดลภาษาขนาดใหญ่ได้รับการพัฒนาจนมีความสามารถในการทำความเข้าใจภาษาธรรมชาติ สรุปลความซับซ้อน และได้ตอบกับผู้ใช้ได้อย่างเป็นธรรมชาติ จึงถูกนำมาประยุกต์ใช้เพื่อทำหน้าที่เป็นระบบผู้ช่วยอัจฉริยะ (AI Assistant) ในการให้ข้อมูลแนวทางการรับมือภัยพิบัติ คู่มือปฏิบัติตน และการวิเคราะห์สถานการณ์เบื้องต้นแก่ผู้ใช้งานผ่านแอปพลิเคชันดี-มายด์[59]

อย่างไรก็ตาม การนำโมเดลภาษาขนาดใหญ่มาใช้งานแบบเดี่ยว (Standalone) ในงานเฉพาะทางยังพบข้อจำกัดเชิงเทคนิคที่สำคัญ ได้แก่ 1) ความรู้ของโมเดลถูกจำกัดไว้เฉพาะข้อมูลที่ผ่านการฝึกฝนทำให้ไม่สามารถอัปเดตข้อมูลภัยพิบัติหรือระเบียบการปฏิบัติงานแบบเวลาจริงได้ 2) ความเสี่ยงต่อการสร้างข้อมูลที่ไม่ถูกต้องหรือที่เรียกว่า AI Hallucination ซึ่งอาจส่งผลกระทบต่อการตัดสินใจในสถานการณ์ฉุกเฉิน และ 3) การขาดความสามารถในการอ้างอิงแหล่งข้อมูลอย่างเป็นทางการได้อย่างแม่นยำ เพื่อแก้ไขข้อจำกัดดังกล่าวและยกระดับความน่าเชื่อถือของระบบผู้ช่วยอัจฉริยะ คณะผู้วิจัยจึงได้นำสถาปัตยกรรมการสร้างข้อความโดยเสริมการค้นคืนข้อมูล (Retrieval-Augmented Generation: RAG) มาประยุกต์ใช้ ซึ่งเป็นการผสมผสานขีดความสามารถในการสร้างภาษาของโมเดลเข้ากับฐานความรู้ภายนอกที่ผ่านการจัดการเชิงความหมาย ช่วยให้ระบบสามารถสืบค้นข้อมูลที่ทันสมัยและตรงประเด็น ก่อนนำข้อมูลดังกล่าวมาสังเคราะห์เป็นคำตอบที่ถูกต้อง สอดคล้องกับสถานการณ์จริงและลดโอกาสเกิดข้อมูลคลาดเคลื่อน ดังรายละเอียดหลักการทำงานและแนวทางการบูรณาการในระบบดี-มายด์ที่นำเสนอในหัวข้อถัดไป[60]

2.7.1 การสร้างข้อความโดยเสริมการค้นคืนข้อมูล (Retrieval-Augmented Generation)

การสร้างข้อความโดยเสริมการค้นคืนข้อมูล หรือ RAG เป็นสถาปัตยกรรมขั้นสูงในกระบวนการประมวลผลภาษาธรรมชาติที่ได้รับการพัฒนาขึ้นเพื่อแก้ไขข้อจำกัดสำคัญของโมเดลภาษาขนาดใหญ่ โดยเฉพาะปัญหาด้านความทันสมัยของข้อมูล และปัญหากระบวนการสร้างข้อมูลเท็จหรือ AI Hallucination โดยทั่วไปโมเดลภาษาจะตอบคำถามโดยอาศัยหน่วยความจำเชิง

พารามิเตอร์ ซึ่งเป็นองค์ความรู้ที่ถูกบันทึกไว้ระหว่างกระบวนการฝึกฝนและจะหยุดนิ่ง ณ ช่วงเวลาที่สิ้นสุดการฝึกฝน ทำให้ไม่สามารถตอบสนองต่อข้อมูลใหม่หรือข้อมูลเฉพาะกลุ่มได้อย่างแม่นยำ[18]

2.7.1.1 หลักการทำงานของ RAG

หลักการพื้นฐานของ RAG คือการผนวกหน่วยความจำแบบนอนพารามิเตอร์ หรือคลังข้อมูลภายนอก เข้ากับขีดความสามารถในการสร้างภาษาของโมเดล โดยกระบวนการทำงานสามารถแบ่งออกเป็น 3 ขั้นตอนหลัก ได้แก่

1) การสืบค้น (Retrieval): ข้อความค้นหา (Query) ของผู้ใช้ จะถูกนำไปใช้ในการสืบค้นเนื้อหาที่เกี่ยวข้องจากแหล่งข้อมูลภายนอก (External Knowledge Source) ในขั้นตอนนี้ query ของผู้ใช้จะถูกแปลงให้กลายเป็นเวกเตอร์ผ่าน embedding model เพื่อนำไปเปรียบเทียบกับเพื่อค้นหาเวกเตอร์ที่มีความคล้ายมากที่สุดกับในเวกเตอร์ดาต้าเบส

2) การเสริมบริบท (Augmentation): query ของผู้ใช้ และเนื้อหาเพิ่มเติมที่ได้มาจากขั้นตอนการ retrieve จะถูกนำไปใส่ใน template ของ prompt

3) การสร้างคำตอบ (Generation): Generate: สดุดท้าย prompt ที่ได้จากการดึงข้อมูลเนื้อหาเสริมมาประกอบจะถูกป้อนเป็น input ให้แก่ LLM ต่อไป[18]

2.7.1.2 การประยุกต์ใช้ RAG ในระบบดี-มายด์

ในการดำเนินงานวิจัยและพัฒนาระบบดี-มายด์ ผู้วิจัยได้นำสถาปัตยกรรม RAG มาประยุกต์ใช้อย่างเต็มรูปแบบเพื่อสร้างระบบผู้ช่วยอัจฉริยะด้านภัยพิบัติ โดยมีรายละเอียดทางเทคนิคดังนี้

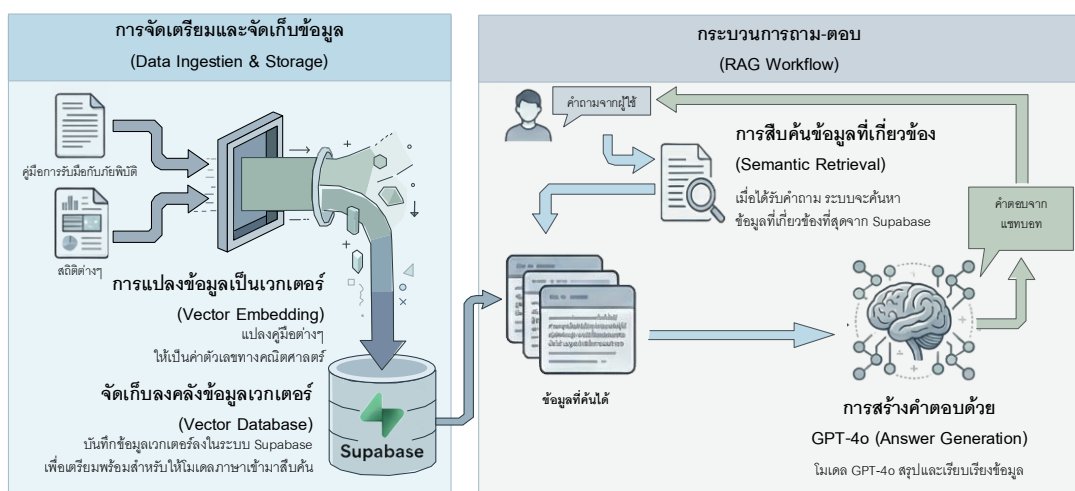
1) การจัดเตรียมและจัดเก็บข้อมูล (Data Ingestion & Storage): ข้อมูลองค์ความรู้เฉพาะทาง อาทิ คู่มือการรับมือภัยพิบัติ สถิติสาธารณภัย และระเบียบวิธีปฏิบัติงานของระบบดี-มายด์ จะถูกแปลงสภาพให้อยู่ในรูปแบบของ “เวกเตอร์” ซึ่งเป็นการแปลงข้อความให้เป็นตัวเลขทางคณิตศาสตร์ที่สามารถวัดค่าความหมายได้ และจัดเก็บลงในฐานข้อมูลซูพาเบส ซึ่งทำหน้าที่เป็นคลังข้อมูลเวกเตอร์ที่มีประสิทธิภาพสูง

2) กระบวนการสืบค้นเชิงความหมาย (Semantic Retrieval): เมื่อมีการนำเข้าคำถาม ระบบจะแปลงคำถามนั้นเป็นเวกเตอร์และคำนวณหาความคล้ายคลึง เมื่อเทียบกับข้อมูลในฐานข้อมูล กระบวนการนี้ช่วยให้ระบบสามารถค้นหาคำตอบได้ตรงตามความหมายที่แท้จริงของผู้ถาม มากกว่าการค้นหาด้วยคำสำคัญแบบดั้งเดิม ทำให้สามารถดึงข้อมูลที่เกี่ยวข้องออกมาได้แม่นยำมากขึ้น

3) การสังเคราะห์คำตอบด้วย LLMs (Answer Synthesis): ข้อมูลที่ผ่านการคัดกรองจะถูกส่งต่อไปยังโมเดลปัญญาประดิษฐ์ เช่น GPT-4o gemini-3-flash-preview เป็นต้น

เพื่อทำหน้าที่สรุปใจความ เชื่อมโยงข้อมูล และเรียบเรียงเป็นคำตอบที่เข้าใจง่ายและสอดคล้องกับสถานการณ์ภัยพิบัติ[19]

การประยุกต์ใช้ RAG ในงานวิจัยนี้ช่วยให้ระบบสามารถให้ข้อมูลที่ถูกต้องและอ้างอิงได้ตามเอกสารโครงการ อีกทั้งยังช่วยแก้ปัญหาความทันสมัยของข้อมูล กล่าวคือ หากมีการเปลี่ยนแปลงนโยบายหรือข้อมูลภัยพิบัติชุดใหม่ ผู้วิจัยสามารถอัปเดตข้อมูลลงในฐานข้อมูลได้ทันทีโดยไม่ต้องทำการฝึกฝนโมเดลใหม่ หรือปรับแต่งโมเดลซึ่งช่วยลดต้นทุนและทรัพยากรในการบำรุงรักษาระบบในระยะยาว ดังแสดงขั้นตอนการทำงานในภาพที่ 2.1



ภาพที่ 2.4 ผังขั้นตอน RAG ของระบบดี-มายด์[19]

2.8 ระบบสารสนเทศภูมิศาสตร์

เทคโนโลยีสารสนเทศมีบทบาทสำคัญอย่างยิ่งต่อการพัฒนาระบบเฝ้าระวังและแจ้งเตือนภัยพิบัติ โดยช่วยเพิ่มขีดความสามารถในการรวบรวม ประมวลผล และเผยแพร่ข้อมูลข่าวสารจากแหล่งข้อมูลที่หลากหลายได้ในลักษณะใกล้เคียงเวลาจริง ระบบเฝ้าระวังภัยพิบัติสมัยใหม่มักมีการบูรณาการเทคโนโลยีขั้นสูงเข้าด้วยกัน อาทิ เครือข่ายอินเทอร์เน็ตของสรรพสิ่ง ระบบตรวจวัดระยะไกล เทคโนโลยีภาพถ่ายดาวเทียม เครือข่ายสื่อสารไร้สาย และการประมวลผลบนระบบคลาวด์ เพื่อสร้างภาพรวมของสถานการณ์ที่ถูกต้องและทันท่วงที ตัวอย่างเช่น การประยุกต์ใช้โอไอที ร่วมกับระบบสารสนเทศภูมิศาสตร์ในการเฝ้าระวังอุทกภัย แผ่นดินไหว หรือดินถล่ม เพื่อสนับสนุนการวางแผนเผชิญเหตุและให้ความช่วยเหลือได้อย่างมีประสิทธิภาพ[6]

2.9 งานวิจัยที่เกี่ยวข้อง

จากการทบทวนงานวิจัยต่างๆบนอินเทอร์เน็ตพบว่า มีงานวิจัยจำนวนมากที่นำเสนอแนวทางการประยุกต์ใช้เทคโนโลยีเซ็นเซอร์ไอโอที และระบบสื่อสารไร้สายเพื่อเพิ่มประสิทธิภาพในการเฝ้าระวังและแจ้งเตือนภัยพิบัติ ตัวอย่างเช่น งานวิจัยที่พัฒนาระบบตรวจวัดระดับน้ำและติดตามโครงสร้างเขื่อนแบบเรียลไทม์ ซึ่งออกแบบให้ระบบสามารถส่งสัญญาณเตือนเมื่อระดับน้ำสูงเกินค่ามาตรฐานหรือมีแนวโน้มที่จะก่อให้เกิดอุทกภัย โดยกระบวนการทำงานมักเริ่มจากการรวบรวมข้อมูลจากเซ็นเซอร์ภาคสนามส่งไปยังฐานข้อมูลบนเว็บหรือคลาวด์ จากนั้นทำการประมวลผลเพื่อส่งการแจ้งเตือนผ่านบริการข้อความสั้น หรือเครือข่ายสื่อสารอื่นๆ ไปยังประชาชนและหน่วยงานที่รับผิดชอบ เพื่อให้สามารถบริหารจัดการความเสี่ยงได้ทันท่วงที[34] โดยสามารถสรุปงานวิจัยที่เกี่ยวข้องได้ดังนี้

2.12.1 Environmental AI-based multi-sensor disaster early warning system

งานวิจัยของ Aadith Nair P. et al. (2025) ได้นำเสนอระบบเฝ้าระวังภัยพิบัติที่ใช้เซ็นเซอร์หลายประเภทร่วมกับปัญญาประดิษฐ์ โดยมีการออกแบบให้ระบบสามารถตรวจวัดตัวแปรด้านสิ่งแวดล้อม เช่น อุณหภูมิ ความชื้น คุณภาพอากาศ และการสั่นสะเทือนของพื้นดิน จากนั้นนำข้อมูลเข้าสู่กระบวนการวิเคราะห์ด้วยโมเดลทางสถิติและ AI เพื่อจำแนกระดับความเสี่ยงของภัยพิบัติแบบอัตโนมัติ ระบบดังกล่าวสามารถประมวลผลได้รวดเร็วและรองรับการทำงานแบบเรียลไทม์เหมาะสำหรับการใช้งานในพื้นที่ที่ต้องการระบบเตือนภัยต้นทุนต่ำและมีประสิทธิภาพ[43]

2.12.2 Domain-Specific Retrieval-Augmented Generation Using Vector Stores, Knowledge Graphs, and Tensor Factorization

งานวิจัยของ Ryan Barron et al. (2024) ได้นำเสนอการพัฒนาระบบ Retrieval-Augmented Generation (RAG) สำหรับงานเฉพาะทาง โดยผสมผสานการทำงานระหว่าง Vector Stores, Knowledge Graphs และเทคนิค Tensor Factorization เพื่อเพิ่มประสิทธิภาพของ Large Language Models (LLMs) ในการตอบคำถามเชิงลึกและลดปัญหา hallucination ของโมเดล ระบบสามารถค้นคืนข้อมูลที่เกี่ยวข้องจากฐานข้อมูลเวกเตอร์และกราฟความรู้ แล้วนำข้อมูลบริบทที่ได้ส่งให้ LLM ใช้ในการสร้างคำตอบที่มีความถูกต้องและสามารถอ้างอิงแหล่งที่มาของข้อมูลได้ นอกจากนี้ งานวิจัยยังแสดงให้เห็นว่าการผสมผสานข้อมูลทั้งแบบโครงสร้าง (structured data) และไม่มีโครงสร้าง (unstructured data) ช่วยเพิ่มความแม่นยำในการตอบคำถามเฉพาะด้าน และลดความจำเป็นในการ fine-tuning โมเดลขนาดใหญ่[48]

2.12.3 Developing real-time IoT-based public safety alert and emergency response systems

Han Zhang et al. (2025) ได้พัฒนาระบบแจ้งเตือนภัยและตอบสนองเหตุฉุกเฉินแบบเรียลไทม์ โดยใช้ IoT ร่วมกับโครงสร้างพื้นฐานบนคลาวด์ ระบบสามารถรวบรวมข้อมูลจากอุปกรณ์ตรวจวัดภาคสนาม เช่น Raspberry Pi และไมโครคอนโทรลเลอร์ และส่งข้อมูลไปยังแพลตฟอร์มส่วนกลางเพื่อวิเคราะห์และแสดงผลผ่านแอปพลิเคชัน นอกจากนี้ยังรองรับการแจ้งเตือนผู้ใช้งานแบบทันที ซึ่งช่วยเพิ่มประสิทธิภาพในการบริหารจัดการเหตุฉุกเฉินในระดับชุมชน[45]

2.12.4 Combination of large language models and portable flood sensors for community flood response

งานวิจัยของ Tsung-Hua Ou et al. (2025) ได้นำเสนอแนวทางการผสานเทคโนโลยีโมเดลภาษาขนาดใหญ่ (Large Language Models: LLMs) เข้ากับเซ็นเซอร์ตรวจวัดน้ำท่วมแบบพกพา โดยระบบสามารถนำข้อมูลจากเซ็นเซอร์มาวิเคราะห์ร่วมกับองค์ความรู้จาก LLM เพื่อให้คำแนะนำแก่ผู้ใช้งานเกี่ยวกับการรับมือสถานการณ์น้ำท่วมแบบอัตโนมัติ แนวทางนี้ช่วยเพิ่มความสามารถของระบบจากการแจ้งเตือนเพียงอย่างเดียว ไปสู่การเป็นผู้ช่วยอัจฉริยะในการตัดสินใจ [46]

2.12.5 Secure cross-layer mobile sensing framework for real-time disaster reporting and visualisation

Rashid Mustafa et al. (2025) ได้นำเสนอกรอบแนวคิดของระบบ mobile sensing สำหรับการรายงานเหตุการณ์ภัยพิบัติแบบเรียลไทม์ โดยเน้นการออกแบบสถาปัตยกรรมแบบหลายชั้น (cross-layer) เพื่อเพิ่มความปลอดภัยและประสิทธิภาพในการรับส่งข้อมูล ระบบเปิดโอกาสให้ผู้ใช้งานสามารถรายงานเหตุการณ์ พร้อมระบุพิกัดและรายละเอียดของภัยพิบัติผ่านแอปพลิเคชันบนอุปกรณ์เคลื่อนที่ และนำข้อมูลดังกล่าวไปแสดงผลบนแผนที่เพื่อใช้ในการวิเคราะห์สถานการณ์[47]

ในด้านการพัฒนาแอปพลิเคชันบนอุปกรณ์เคลื่อนที่ มีการศึกษาที่มุ่งเน้นการพัฒนาแพลตฟอร์มสำหรับการรายงานเหตุและการรับแจ้งเตือนภัยอย่างรวดเร็ว โดยเปิดโอกาสให้ผู้ใช้งานมีส่วนร่วมในการส่งข้อมูลพิกัดสถานที่และประเภทของภัยพิบัติเข้าสู่ระบบ ข้อมูลเหล่านี้จะถูกนำไปแสดงผลในแอปพลิเคชันเพื่อช่วยให้ผู้ใช้รายอื่นสามารถหลีกเลี่ยงเส้นทางหรือพื้นที่เสี่ยงภัยได้ นอกจากนี้ จากการสังเคราะห์องค์ความรู้เกี่ยวกับแอปพลิเคชันเตือนภัยที่มีประสิทธิภาพ พบว่าควรประกอบด้วยฟังก์ชันสำคัญ 4 ประการ ได้แก่ 1) ระบบแจ้งเตือนแบบระบุพิกัดตำแหน่ง 2) การแสดงผลแผนที่พื้นที่เสี่ยงภัย 3) การให้ความรู้เกี่ยวกับการเตรียมความพร้อมรับมือภัยพิบัติ และ 4) ช่องทางสำหรับการรายงานเหตุฉุกเฉินโดยผู้ใช้งาน